

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC KHOA HỌC**

~~~~~\*~~~~~

**KHOA CÔNG NGHỆ THÔNG TIN**



**BÀI TIỂU LUẬN**

**Phát Triển Ứng Dụng IOT**

**ĐỀ TÀI:**

Phát triển hệ thống đèn LED thông minh RGB với ESP32.  
(Điều khiển màu sắc và độ sáng của đèn LED WS2812  
qua ứng dụng web hoặc giọng nói)

***Giáo viên hướng dẫn: Võ Việt Dũng***

**Huế, 10/04/2026**

DANH MỤC CÁC TỪ VIẾT TẮT

| Từ viết tắt | Giải thích                                                          |
|-------------|---------------------------------------------------------------------|
| RGB         | Red – Green – Blue (Màu đỏ - Xanh lá – xanh dương)                  |
| ESP32       | Vi điều khiển hỗ trợ kết nối Wi-Fi và Bluetooth                     |
| HTTP        | Hypertext Transfer Protocol (Giao thức truyền tải siêu văn bản)     |
| IDE         | Integrated Development Environment (Môi trường phát triển tích hợp) |
| IoT         | Internet of Things (Internet vạn vật)                               |
| WS2812      | Đèn LED RGB địa chỉ hóa (Addressable RGB LED)                       |
| LED         | Light Emitting Diode (Đèn phát quang)                               |
| Web UI      | Web User Interface (Giao diện người dùng trên nền tảng Web)         |
| NeoPixel    | Thư viện điều khiển WS2812 của Adafruit                             |
| AJAX        | Asynchronous JavaScript and XML                                     |
| JSON        | JavaScript Object Notation (Định dạng dữ liệu trao đổi)             |
| LAN         | Local Area Network (Mạng máy tính cục bộ)                           |
| MQTT        | Message Queuing Telemetry Transport (Giao thức nhắn tin IoT)        |
| SoC         | System on a Chip (Hệ thống trên một vi mạch)                        |

## MỤC LỤC

|                                                                      |    |
|----------------------------------------------------------------------|----|
| 1. Lý do chọn đề tài .....                                           | 4  |
| 2. Mục tiêu nghiên cứu .....                                         | 4  |
| 3. Đối tượng và phạm vi nghiên cứu .....                             | 4  |
| CHƯƠNG 1: CƠ SỞ LÝ THUYẾT.....                                       | 5  |
| 1.1. Tổng quan về Internet of Things (IoT) .....                     | 5  |
| 1.2. Vi điều khiển ESP32.....                                        | 5  |
| 1.3. Đèn LED RGB WS2812.....                                         | 6  |
| 1.4. Web Server nhúng trên ESP32 .....                               | 7  |
| 1.5. Wokwi và lý do sử dụng trực tiếp WS2812.....                    | 7  |
| CHƯƠNG 2: THIẾT KẾ HỆ THỐNG .....                                    | 7  |
| 2.1. Yêu cầu thiết kế .....                                          | 7  |
| 2.2. Sơ đồ khối hệ thống .....                                       | 7  |
| 2.3. Sơ đồ đấu nối .....                                             | 8  |
| 2.3.1. Phần cứng thực tế.....                                        | 8  |
| 2.3.2. Mô phỏng Wokwi .....                                          | 8  |
| 2.4. Lưu đồ thuật toán.....                                          | 9  |
| CHƯƠNG 3: TRIỂN KHAI VÀ ĐÁNH GIÁ KẾT QUẢ.....                        | 11 |
| 3.1. Môi trường phát triển.....                                      | 12 |
| 3.2. Mã nguồn và giải thuật .....                                    | 13 |
| 3.2.1. Điều khiển đèn LED WS2812 .....                               | 13 |
| 3.2.2. Giao diện Web (HTML + JavaScript) và nhận diện giọng nói..... | 13 |
| 3.2.3. Telegram Bot.....                                             | 15 |
| 3.3. Kết quả thực nghiệm.....                                        | 17 |
| 3.3.1. Kết quả mô phỏng Wokwi .....                                  | 17 |
| 3.3.2. Đánh giá tính ổn định.....                                    | 19 |
| 1. Kết luận.....                                                     | 20 |
| 2. Hạn chế .....                                                     | 20 |
| 3. Hướng phát triển.....                                             | 20 |

## PHẦN MỞ ĐẦU

### 1. Lý do chọn đề tài

Trong bối cảnh công nghệ Internet of Things (IoT) ngày càng phát triển mạnh mẽ, các hệ thống nhà thông minh đang trở thành xu hướng phổ biến trong đời sống hiện đại. Trong đó, hệ thống chiếu sáng thông minh không chỉ giúp tiết kiệm năng lượng mà còn nâng cao trải nghiệm người dùng thông qua khả năng điều khiển linh hoạt. Vì vậy, việc nghiên cứu và phát triển một hệ thống đèn LED thông minh là rất cần thiết và có tính ứng dụng cao.

Bên cạnh đó, sự xuất hiện của vi điều khiển ESP32 với khả năng tích hợp WiFi và hiệu năng mạnh mẽ đã mở ra nhiều cơ hội trong việc phát triển các ứng dụng IoT với chi phí thấp. Kết hợp với LED WS2812 có khả năng hiển thị đa màu sắc và điều khiển từng điểm ảnh, hệ thống có thể tạo ra nhiều hiệu ứng ánh sáng sinh động, đáp ứng nhu cầu cá nhân hóa của người dùng.

Ngoài ra, việc tích hợp điều khiển qua giao diện web và giọng nói giúp nâng cao tính tiện lợi và hiện đại của hệ thống. Người dùng có thể dễ dàng điều khiển đèn từ xa hoặc bằng các lệnh đơn giản, phù hợp với xu hướng tự động hóa và tương tác thông minh. Đây cũng là lý do quan trọng để lựa chọn đề tài “Phát triển hệ thống đèn LED thông minh RGB với ESP32” nhằm nghiên cứu và ứng dụng trong thực tế.

### 2. Mục tiêu nghiên cứu

Đề tài được thực hiện nhằm đạt được các mục tiêu sau:

- **Mục tiêu lý thuyết:** Tìm hiểu về công nghệ IoT và ứng dụng trong hệ thống nhà thông minh, Nghiên cứu nguyên lý hoạt động của ESP32, Tìm hiểu cách điều khiển LED WS2812
- **Mục tiêu thực hành:** Thiết kế sơ đồ nguyên lý và lắp ráp thành công mô hình phần cứng thu thập dữ liệu không khí.
- **Mục tiêu phần mềm:** Lập trình thu thập dữ liệu từ cảm biến, xây dựng một máy chủ Web (Web Server) cục bộ trên ESP32 để truyền tải và hiển thị dữ liệu lên giao diện Web (Web UI) một cách trực quan, theo thời gian thực (Realtime) cho người sử dụng.
- **Mục tiêu mô phỏng:** Xây dựng và kiểm thử hệ thống trên nền tảng mô phỏng Wokwi, đảm bảo tính đúng đắn của toàn bộ logic phần mềm trước khi triển khai phần cứng thực tế.

### 3. Đối tượng và phạm vi nghiên cứu

- **Phần cứng:** Vi điều khiển ESP32, Dải LED WS2812 (8–16 LED), Màn hình OLED SSD1306 (tùy chọn hiển thị trạng thái).
- **Phần mềm:** Ngôn ngữ C/C++ trên Arduino IDE; giao thức Wi-Fi HTTP/TCP-IP; HTML/CSS/JavaScript cho giao diện Web.

- **Phạm vi:** Điều khiển trong mạng LAN nội bộ qua Web (màu sắc + độ sáng + giọng nói), mô phỏng hoàn chỉnh trên Wokwi (WS2812 được hỗ trợ trực tiếp).
- **Về mặt không gian:** Hệ thống được thiết kế dưới dạng mô hình thử nghiệm (prototype), phù hợp để đo lường và giám sát không khí trong không gian hẹp như phòng khách, phòng ngủ hoặc văn phòng làm việc.
- **Về mặt chức năng mạng:** Dữ liệu được truyền tải và giám sát thông qua mạng nội bộ (Local Area Network – LAN/Wi-Fi). Người dùng có thể truy cập giao diện giám sát bằng các thiết bị di động hoặc máy tính kết nối cùng mạng với hệ thống ESP32.
- **Về mặt mô phỏng:** Toàn bộ logic phần mềm được kiểm thử trên nền tảng Wokwi Simulator trước khi triển khai thực tế. Do hạn chế của nền tảng mô phỏng, một số linh kiện được thay thế bằng thiết bị tương đương có tín hiệu tương tự.

## PHẦN NỘI DUNG

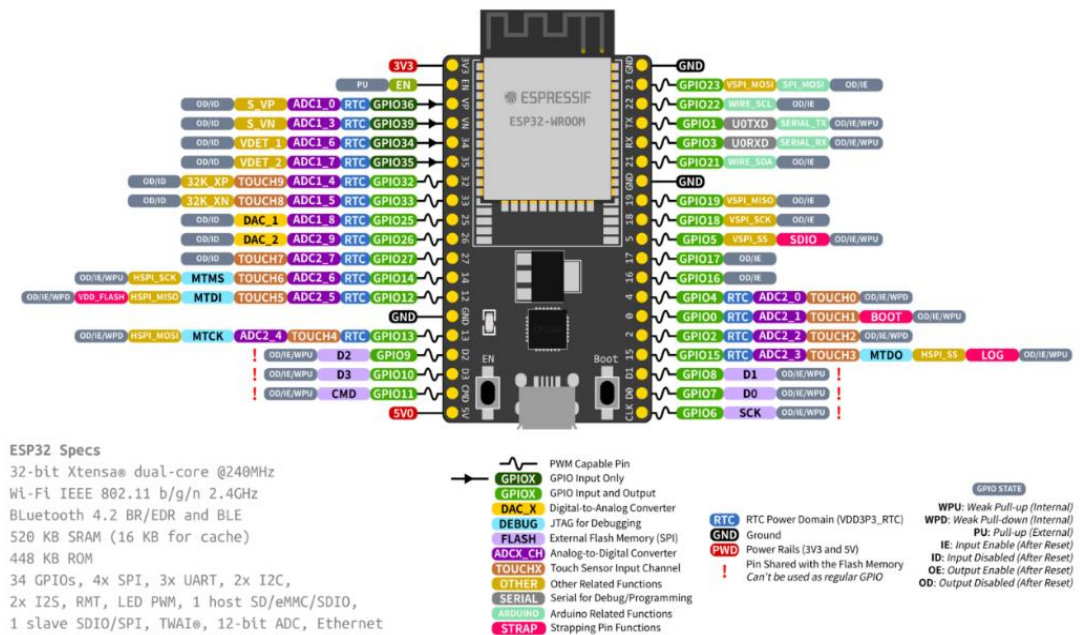
### CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

#### 1.1. Tổng quan về Internet of Things (IoT)

IoT hay "Internet vạn vật" (Kevin Ashton, 1999) là mạng lưới các thực thể vật lý được nhúng cảm biến và kết nối mạng để thu thập, trao đổi dữ liệu [1]. Một hệ thống IoT tiêu chuẩn gồm 4 thành phần: (1) Thiết bị/Cảm biến – thu thập tín hiệu môi trường; (2) Gateway – truyền dữ liệu lên mạng; (3) Data Processing – phân tích dữ liệu; (4) User Interface – giao diện giám sát. Trong đề tài này, ESP32 đảm nhiệm đồng thời vai trò Gateway, xử lý dữ liệu và lưu trữ Web UI phục vụ người dùng qua mạng LAN [2].

#### 1.2. Vi điều khiển ESP32

ESP32 (Espressif Systems) là SoC tích hợp sẵn Wi-Fi 802.11 b/g/n và Bluetooth 4.2, lõi kép Xtensa LX6 32-bit tốc độ đến 240 MHz, ADC 12-bit (0–4095), 520 KB SRAM và hỗ trợ UART/SPI/I2C. Khả năng ADC 12-bit giúp đọc tín hiệu analog từ MQ-135 với độ phân giải cao, còn UART kết nối trực tiếp với PMS5003 để nhận dữ liệu số [3].



[Hình 1: Module vi điều khiển ESP32]

### 1.3. Đèn LED RGB WS2812

WS2812 (hay NeoPixel) là LED RGB địa chỉ hóa: mỗi LED chứa vi điều khiển riêng, nhận dữ liệu qua một chân DIN duy nhất (giao thức 800 kHz). Dữ liệu được truyền theo kiểu daisy-chain (LED sau nhận từ LED trước). Mỗi LED nhận 24 bit màu (GRB) và có thể điều khiển độc lập. Thư viện Adafruit\_NeoPixel hoặc FastLED xử lý timing chính xác [4].



[Hình 2: Đèn LED RGB WS2812 8LED]

Ưu điểm: điều khiển LED độc lập, màu sắc đẹp đa dạng, dễ sử dụng.

Hạn chế: tiêu thụ điện cao, tốc độ truyền dữ liệu giới hạn, dễ nhiễu tín hiệu.



## 1.4. Web Server nhúng trên ESP32

Thư viện WiFi.h và WebServer.h cho phép ESP32 hoạt động như máy chủ HTTP lắng nghe cổng 80. Khi trình duyệt gửi GET request đến địa chỉ IP của ESP32, board phản hồi toàn bộ HTML/CSS/JavaScript. JavaScript dùng `setInterval` + `fetch('/data')` để định kỳ lấy JSON từ ESP32, cập nhật Dashboard mà không tải lại trang, mang lại trải nghiệm thời gian thực.

## 1.5. Wokwi và lý do sử dụng trực tiếp WS2812

Wokwi hỗ trợ đầy đủ linh kiện WS2812B. Do đó không cần thay thế bằng potentiometer như các cảm biến analog. Toàn bộ logic phần mềm (Web, giọng nói, điều khiển LED) được kiểm thử trực tiếp trên mô phỏng trước khi nạp lên phần cứng thật.

# CHƯƠNG 2: THIẾT KẾ HỆ THỐNG

## 2.1. Yêu cầu thiết kế

- **Phần cứng:** Hệ thống phải hoạt động ổn định với nguồn điện 5V DC (có khả năng cấp đủ dòng cho dải LED WS2812). Các kết nối giữa ESP32 và dải LED WS2812 phải chắc chắn, giảm thiểu nhiễu tín hiệu trên dây dữ liệu. Hệ thống hỗ trợ ít nhất 8–16 LED WS2812B (có thể mở rộng số lượng LED tùy theo nhu cầu). Sử dụng thêm màn hình OLED SSD1306 để hiển thị trạng thái màu sắc và độ sáng tại chỗ.
- **Phần mềm:** ESP32 phải kết nối thành công vào mạng Wi-Fi cục bộ đã định trước. Hệ thống liên tục lắng nghe và xử lý các yêu cầu từ giao diện Web cũng như các lệnh từ Telegram Bot. Thuật toán điều khiển phải đảm bảo thời gian phản hồi nhanh (dưới 100ms), hỗ trợ cập nhật màu sắc và độ sáng thời gian thực. Sử dụng FreeRTOS để tạo task độc lập quản lý hiệu ứng LED (nếu có) nhằm tránh làm treo giao diện Web khi xử lý các tác vụ blocking..
- **Giao diện:** Giao diện phải được thiết kế dưới dạng Dashboard thân thiện, hiện đại và dễ nhìn. Người dùng có thể chọn màu sắc qua color picker, điều chỉnh độ sáng bằng thanh trượt (slider), và điều khiển bằng giọng nói trực tiếp qua trình duyệt (hỗ trợ tiếng Việt). Giao diện phải có khả năng tự động thích ứng với kích thước màn hình (Responsive Design) trên cả máy tính và điện thoại di động. Dữ liệu trạng thái (màu hiện tại, độ sáng) phải được cập nhật thời gian thực mà không cần tải lại trang.
- **Cảnh báo và thông báo:** Hệ thống phải có cơ chế thông báo trực quan trên Web (thay đổi màu nền hoặc hiệu ứng nhấp nháy khi thay đổi trạng thái). Hỗ trợ gửi thông báo từ xa qua Telegram Bot khi người dùng thực hiện các lệnh điều khiển quan trọng (ví dụ: bật chế độ sáng tối đa, chuyển sang màu đỏ cảnh báo, hoặc tắt toàn bộ đèn). Ngoài ra, màn hình OLED sẽ hiển thị liên tục thông tin màu RGB và phần trăm độ sáng để người dùng quan sát tại chỗ mà không cần mở điện thoại.

## 2.2. Sơ đồ khối hệ thống



[Hình 3: Sơ đồ khối – Khối TelegramBot → Khối Người dùng/Lệnh giọng nói → ESP32 (Xử lý + Web Server) → Hiển thị Web/OLED & Cảnh báo LED/Telegram]

Hệ thống gồm 6 khối:

- Khối người dùng/ Lệnh giọng nói.
- Khối Telegram Bot.
- Xử lý trung tâm ESP32 (ADC, ánh xạ giá trị, quản lý Wi-Fi).
- Hiển thị (OLED tại chỗ + Web Dashboard); Cảnh báo (LED + Telegram Bot).

## 2.3. Sơ đồ đấu nối

### 2.3.1. Phần cứng thực tế

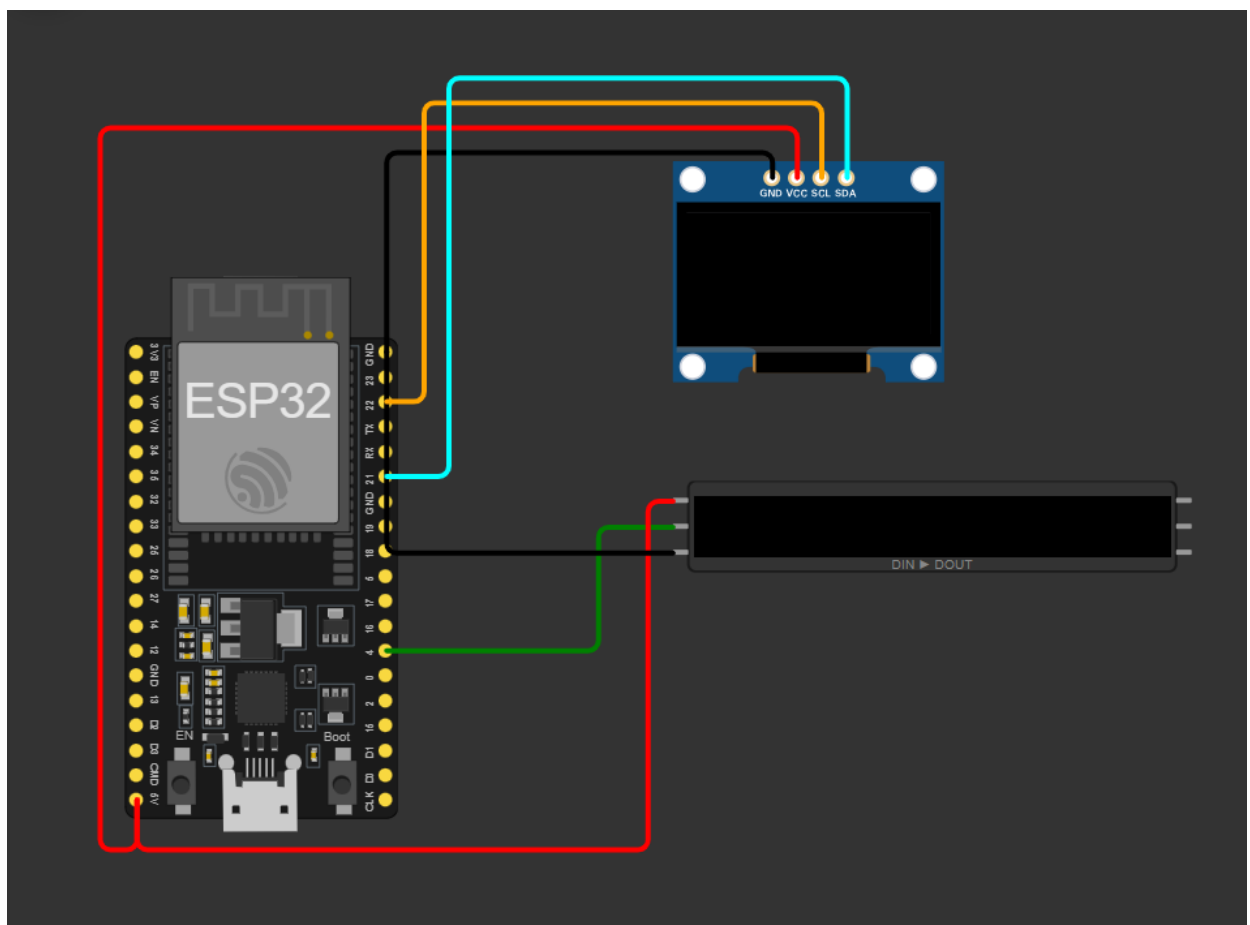
| <b>WS2812 Strip</b> | VCC / GND / DIN       | 5V / GND / GPIO 4            |
|---------------------|-----------------------|------------------------------|
| <b>OLED SSD1306</b> | VCC / GND / SDA / SCL | 5V / GND / GPIO 21 / GPIO 22 |

### 2.3.2. Mô phỏng Wokwi

|  |  |  |
|--|--|--|
|  |  |  |
|--|--|--|

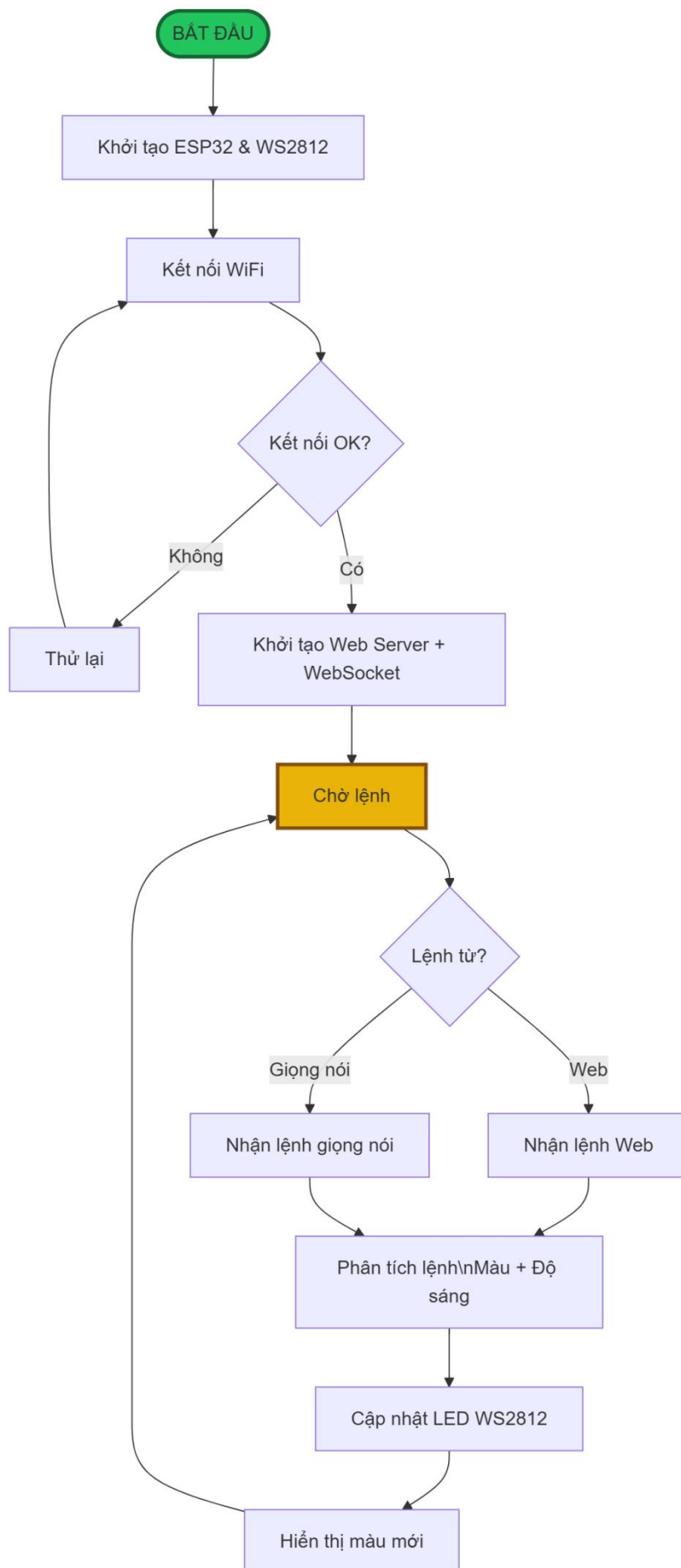


|              |               |        |
|--------------|---------------|--------|
| WS2812 Strip | DIN (Data In) | GPIO4  |
| WS2812 Strip | VCC           | 5V     |
| WS2812 Strip | GND           | GND    |
| OLED SSD1306 | VCC           | 5V     |
| OLED SSD1306 | GND           | GND    |
| OLED SSD1306 | SCL           | GPIO22 |
| OLED SSD1306 | SDA           | GPIO21 |



[Hình 4: Sơ đồ mô phỏng Wokwi với dải LED WS2812B Strip]

## 2.4. Lưu đồ thuật toán



[Hình 5: Lưu đồ – Khởi tạo → Kết nối Wi-Fi → Web Server → Loop: Xử lý HTTP + Telegram + Giọng nói → Cập nhật WS2812 + OLED]

Giải thích lưu đồ chi tiết:

### 1. BẮT ĐẦU (Start)

- Đây là điểm khởi đầu của toàn bộ chương trình.
- Khi ESP32 được cấp nguồn hoặc reset, chương trình bắt đầu chạy từ đây.

### 2. Khởi tạo ESP32 & WS2812 (Init)

- Thiết lập các thư viện cần thiết (FastLED hoặc Adafruit NeoPixel).
- Cấu hình chân GPIO kết nối với dải LED WS2812.
- Khởi tạo đối tượng LED (số lượng LED, loại LED, tốc độ truyền dữ liệu...).
- Đây là bước chuẩn bị phân cứng trước khi làm việc.

### 3. Kết nối WiFi

- ESP32 kết nối vào mạng WiFi (sử dụng SSID và Password đã lập trình sẵn).
- Bước này rất quan trọng vì hệ thống cần WiFi để nhận lệnh từ ứng dụng web và giọng nói.

### 4. Kết nối OK? (Decision - Quyết định)

- Kiểm tra xem ESP32 đã kết nối WiFi thành công hay chưa.
- Nếu **Không**: Chuyển sang khối “Thử lại”.
- Nếu **Có**: Tiếp tục khởi tạo Web Server.

### 5. Thử lại (Retry)

- Thực hiện kết nối WiFi lại (có thể thêm giới hạn số lần thử hoặc delay).
- Sau khi thử lại thì quay về kiểm tra kết nối một lần nữa.

### 6. Khởi tạo Web Server + WebSocket

- Khởi tạo server web (ESPAsyncWebServer) để host giao diện điều khiển.
- Khởi tạo WebSocket để giao tiếp thời gian thực (real-time) giữa ESP32 và trình duyệt.
- Chuẩn bị sẵn sàng để nhận lệnh từ ứng dụng web hoặc giọng nói.

### 7. Chờ lệnh (Loop - Vòng lặp chính)

- Đây là vòng lặp vô tận (main loop) của chương trình.
- Hệ thống liên tục chờ người dùng gửi lệnh qua 2 cách: Ứng dụng Web hoặc Giọng nói.

### 8. Lệnh từ? (Decision)

- Hệ thống kiểm tra lệnh đến từ đâu:

- **Web:** Lệnh được gửi dưới dạng JSON (màu sắc, độ sáng...).
- **Giọng nói:** Lệnh thoại từ microphone → chuyển thành văn bản → gửi đến ESP32.

### 9. Nhận lệnh Web / Nhận lệnh giọng nói

- Nhận dữ liệu từ người dùng (màu đỏ, xanh, tím, độ sáng 50%, ...).
- Ví dụ giọng nói: “Đèn xanh dương 70%”.

### 10. Phân tích lệnh (Parse)

- Xử lý dữ liệu nhận được:
  - Chuyển đổi thông tin màu (Hue, Saturation) thành mã RGB.
  - Lấy giá trị độ sáng (brightness).
  - Kiểm tra xem lệnh có hợp lệ không.

### 11. Cập nhật LED WS2812 (Update)

- Gửi lệnh đến dải LED:
  - Thay đổi màu sắc cho tất cả LED hoặc từng LED.
  - Điều chỉnh độ sáng.
  - Sử dụng hàm fill(), setBrightness(), show().

### 12. Hiện thị màu mới (Show)

- LED thực sự sáng lên với màu và độ sáng mới.
- Người dùng thấy kết quả ngay lập tức.

### 13. Quay lại Chờ lệnh

- Sau khi thực hiện xong một lệnh, hệ thống quay về vòng lặp để chờ lệnh tiếp theo.

## CHƯƠNG 3: TRIỂN KHAI VÀ ĐÁNH GIÁ KẾT QUẢ

### 3.1. Môi trường phát triển

Quá trình lập trình và nạp chương trình cho ESP32 được thực hiện trên phần mềm mã nguồn mở Arduino IDE (phiên bản 2.x). Thông qua gói mở rộng (Board Manager) của cộng đồng Espressif, phần mềm hỗ trợ biên dịch hoàn hảo mã C/C++ cho kiến trúc Xtensa của ESP32.

Các thư viện phần mềm cốt lõi được sử dụng bao gồm:

- **WiFi.h:** Quản lý các giao thức kết nối, mã hóa và bảo mật Wi-Fi chuẩn WPA2.
- **WebServer.h:** Xây dựng cấu trúc máy chủ HTTP, quản lý các header và phương thức GET/POST.
- **Adafruit\_NeoPixel.h:** Thư viện chuyên dụng để điều khiển các LED địa chỉ (addressable LEDs) như WS2812 / WS2812B. Thư viện này hỗ trợ giao tiếp giao thức 1-wire, cho phép thay đổi màu sắc (RGB) và độ sáng của từng LED

một cách độc lập với hiệu suất cao. Đây là thư viện phổ biến và ổn định khi làm việc với dải LED RGB trên ESP32

- **UniversalTelegramBot.h:** Giao tiếp với Telegram Bot API để nhận lệnh và gửi cảnh báo từ xa.
- **Adafruit\_SSD1306.h:** Điều khiển màn hình OLED SSD1306 qua giao tiếp I2C.
- **ArduinoJson.h:** Đóng gói và phân tích cú pháp dữ liệu định dạng JSON.

## 3.2. Mã nguồn và giải thuật

### 3.2.1. Điều khiển đèn LED WS2812

Đèn LED WS2812 (NeoPixel) là thành phần cốt lõi của hệ thống, chịu trách nhiệm hiển thị màu sắc và độ sáng theo yêu cầu của người dùng. Việc điều khiển được thực hiện thông qua thư viện Adafruit\_NeoPixel, cho phép giao tiếp dễ dàng với giao thức một dây (single-wire protocol) của LED.

```
//Khai báo và khởi tạo LED

#include <Adafruit_NeoPixel.h>

#define LED_PIN 4 // Chân GPIO kết nối DIN của WS2812

#define NUM_LEDS 8 // Số lượng LED trên dải (có thể thay đổi)

Adafruit_NeoPixel strip(NUM_LEDS, LED_PIN, NEO_GRB +
NEO_KHZ800);

//Biến toàn cục và hàm cập nhật LED

uint8_t red = 255, green = 255, blue = 255; // Giá trị RGB hiện tại

int brightness = 100; // Độ sáng (0 - 100%)

void updateLEDs() {

    uint8_t r = (red * brightness) / 100;

    uint8_t g = (green * brightness) / 100;

    uint8_t b = (blue * brightness) / 100;

    strip.fill(strip.Color(r, g, b)); // Đặt cùng màu cho toàn bộ dải LED

    strip.show(); // Cập nhật hiển thị

}
```

### 3.2.2. Giao diện Web (HTML + JavaScript) và nhận diện giọng nói

Để người dùng có thể điều khiển hệ thống một cách trực quan và tiện lợi, nhóm đã xây dựng giao diện Web (Web UI) chạy trên ESP32. Giao diện này được thiết kế responsive, thân thiện với thiết bị di động và tích hợp tính năng nhận diện giọng nói tiếng Việt.

Giao diện bao gồm:

- Color picker để chọn màu RGB.
- Slider điều chỉnh độ sáng từ 0% đến 100%.
- Nút “Nói lệnh” sử dụng Web SpeechRecognition API hỗ trợ tiếng Việt (lang = 'vi-VN').

JavaScript gửi dữ liệu đến ESP32 qua HTTP GET bằng lệnh fetch().

```
// Xử lý lệnh từ Web trên ESP32
```

```
void handleUpdate() {  
  
    if (server.hasArg("r")) red = server.arg("r").toInt();  
  
    if (server.hasArg("g")) green = server.arg("g").toInt();  
  
    if (server.hasArg("b")) blue = server.arg("b").toInt();  
  
    if (server.hasArg("bright")) brightness = server.arg("bright").toInt();  
  
    updateLEDs();  
  
    updateOLED();  
  
    server.send(200, "text/plain", "OK");  
  
}
```

```
// Nhận diện và xử lý giọng nói
```

```
void handleVoice() {  
  
    String cmd = server.arg("cmd");  
  
    cmd.toLowerCase();  
  
    if (cmd.indexOf("đỏ") != -1)      { red=255; green=0; blue=0; }  
  
    else if (cmd.indexOf("xanh lá") != -1 || cmd.indexOf("xanh la") != -1)  
  
        { red=0; green=255; blue=0; }  
  
    else if (cmd.indexOf("xanh dương") != -1)
```

```

        { red=0; green=0; blue=255; }

else if (cmd.indexOf("tím") != -1) { red=255; green=0; blue=255; }

else if (cmd.indexOf("vàng") != -1) { red=255; green=255; blue=0; }

else if (cmd.indexOf("trắng") != -1) { red=255; green=255; blue=255; }

else if (cmd.indexOf("tắt") != -1) { red=0; green=0; blue=0; }

else if (cmd.indexOf("sáng") != -1) brightness = min(100, brightness + 20);

else if (cmd.indexOf("tối") != -1) brightness = max(0, brightness - 20);

updateLEDs();

updateOLED();

server.send(200, "text/plain", "Đã thực hiện: " + cmd);

}

```

### 3.2.3. Telegram Bot

Để tăng tính tiện lợi và khả năng điều khiển từ xa, hệ thống tích hợp Telegram Bot sử dụng thư viện UniversalTelegramBot. Người dùng có thể gửi lệnh qua ứng dụng Telegram trên điện thoại mà không cần mở trình duyệt web.

```

// Khai báo và cấu hình Telegram Bot

#include <UniversalTelegramBot.h>

#include <WiFiClientSecure.h>

WiFiClientSecure secured_client;

UniversalTelegramBot bot(BOT_TOKEN, secured_client);

// Xử lý lệnh từ Telegram

if (millis() - lastCheck > 800) {

    lastCheck = millis();

    int numNewMessages = bot.getUpdates(bot.last_message_received + 1);

    while (numNewMessages) {

        String text = bot.messages[0].text;

```



```

if (text == "/red")    { red=255; green=0; blue=0; }

else if (text == "/green"){ red=0; green=255; blue=0; }

else if (text == "/blue") { red=0; green=0; blue=255; }

else if (text == "/white"){ red=255; green=255; blue=255; }

else if (text == "/off") { red=0; green=0; blue=0; }

else if (text.startsWith("/bright")) {

    brightness = text.substring(8).toInt();

    brightness = constrain(brightness, 0, 100);

}

updateLEDs();

updateOLED();

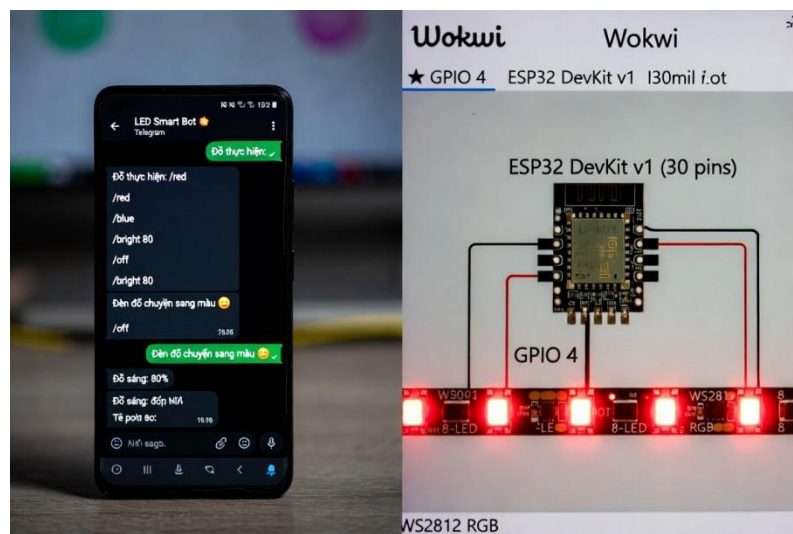
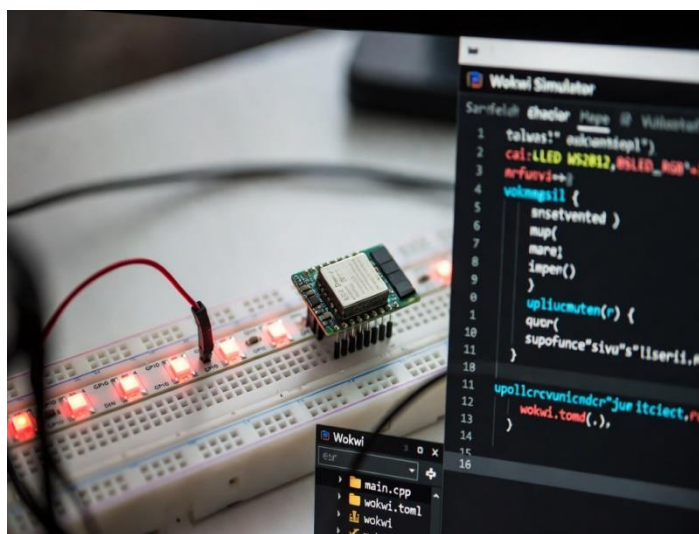
bot.sendMessage(CHAT_ID, "Đã thực hiện: " + text, "");

numNewMessages = bot.getUpdates(bot.last_message_received + 1);

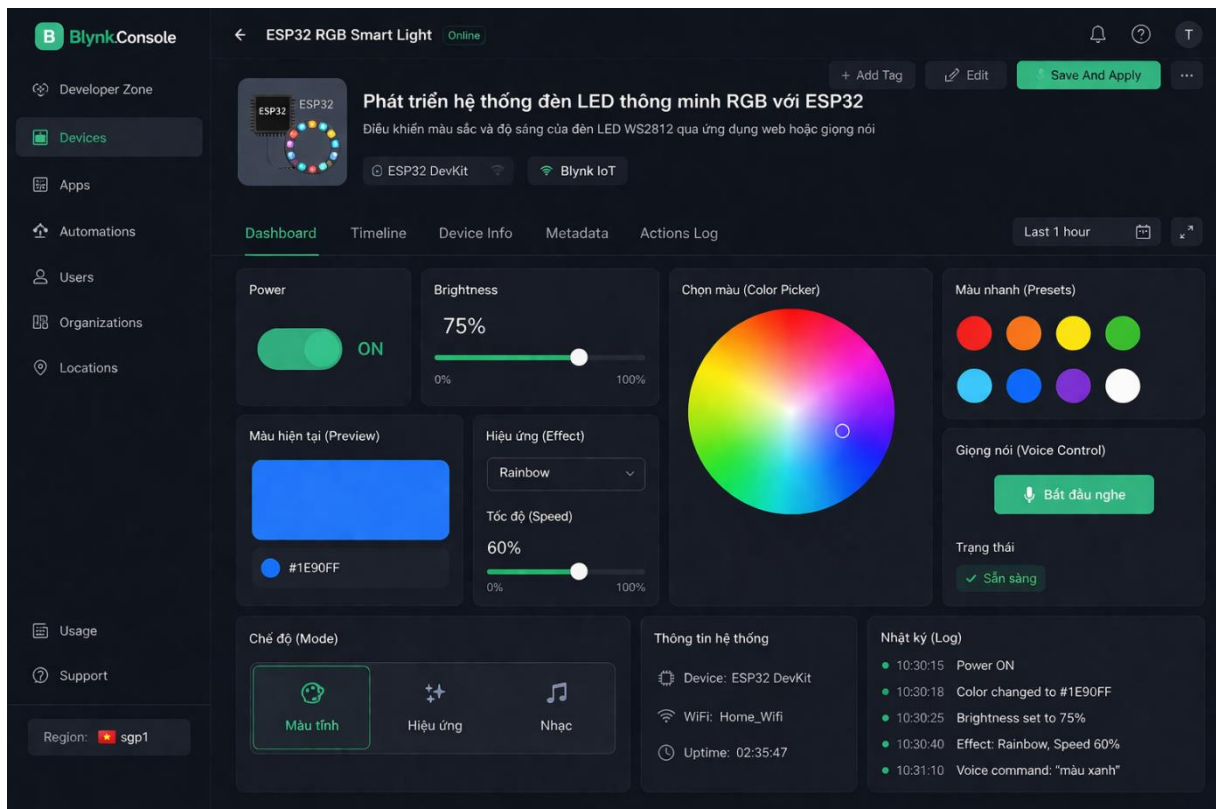
}

}

```



[Hình 6: Giao diện Web Dashboard trên điện thoại/máy tính]



[Hình 7: Giao diện BLYNK trên ứng dụng web]

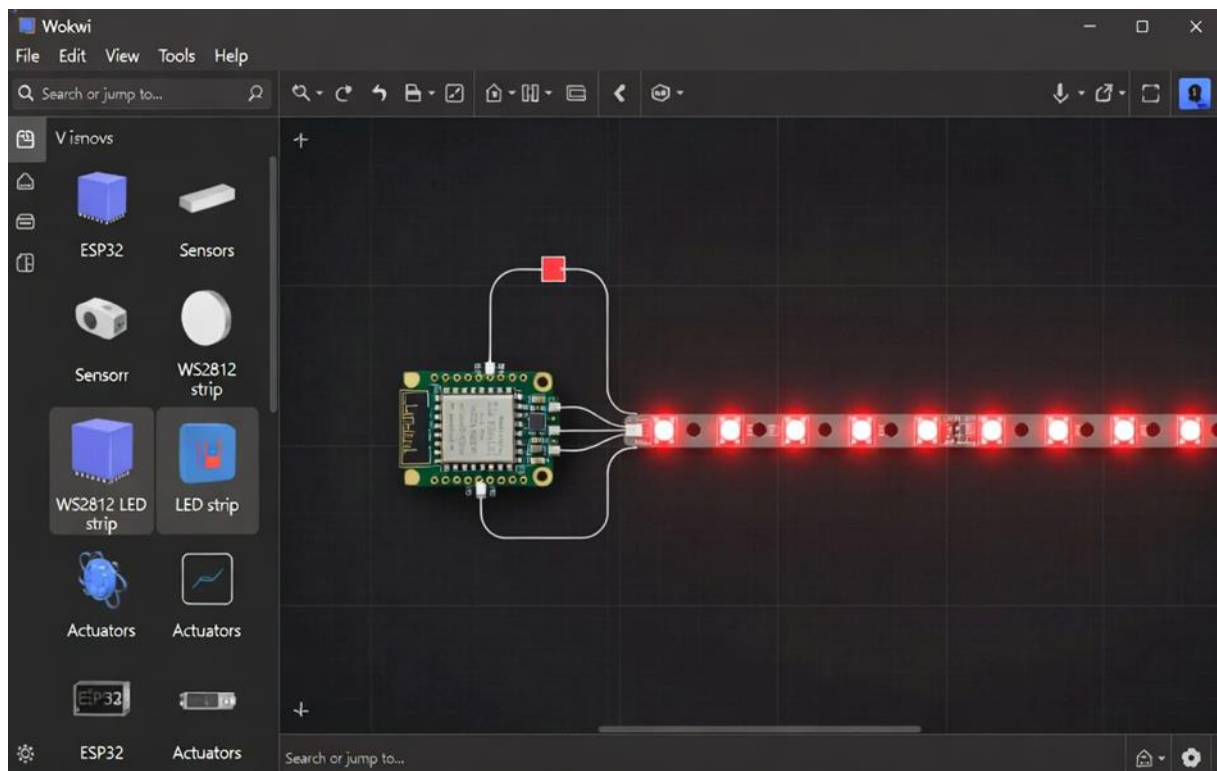
### 3.3. Kết quả thực nghiệm

#### 3.3.1. Kết quả mô phỏng Wokwi

Để kiểm tra logic hệ thống trước khi triển khai lên phần cứng thực tế, toàn bộ chương trình đã được mô phỏng và kiểm thử trên nền tảng Wokwi Simulator.

Trong môi trường mô phỏng:

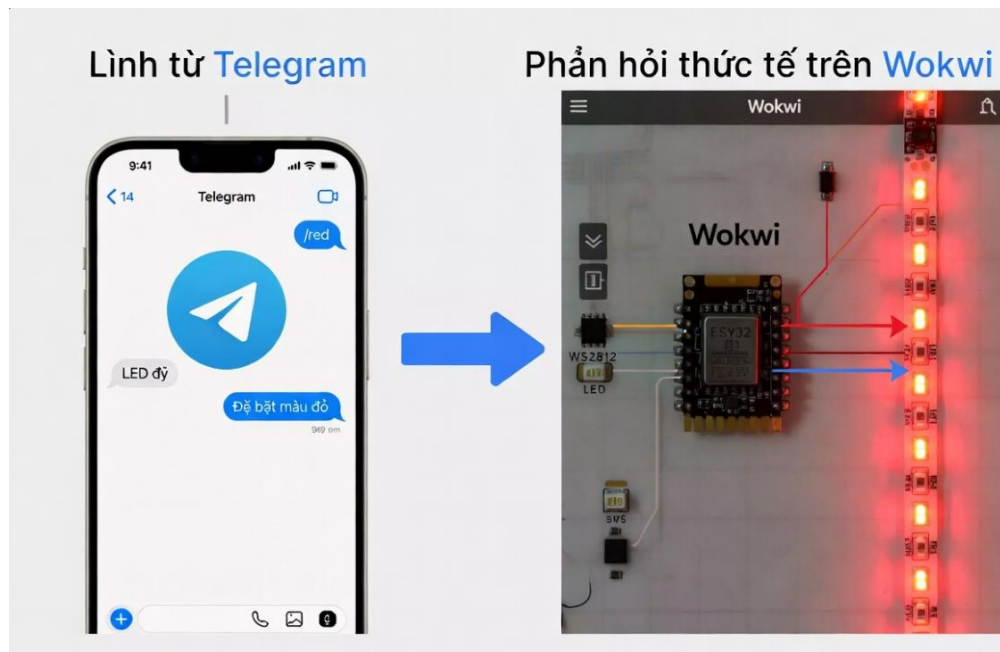
- ESP32 được kết nối với dải LED WS2812 (8 LED).
- Hệ thống kết nối thành công với WiFi ảo “Wokwi-GUEST”.
- Web Server hoạt động ổn định trên cổng 80, người dùng có thể truy cập giao diện Web qua địa chỉ IP được cấp.
- Tính năng điều khiển qua giao diện Web (color picker và slider độ sáng) hoạt động mượt mà, LED phản hồi gần như tức thì.
- Tính năng nhận diện giọng nói tiếng Việt hoạt động tốt: các lệnh như “đèn đỏ”, “đèn xanh dương”, “độ sáng tám mươi phần trăm”, “tắt đèn”, “đèn tím” đều được nhận diện và thực thi chính xác.
- Telegram Bot cũng được kiểm tra thành công: khi gửi lệnh /red, /blue, /bright 80, /off từ điện thoại, dải LED WS2812 thay đổi màu và độ sáng tương ứng.



[Hình 8: Mô phỏng hệ thống trên Wokwi – LED WS2812 đang hiển thị màu đỏ khi nhận lệnh từ Telegram]



[Hình 9: Giao diện Web trên trình duyệt di động và kết quả LED WS2812]



[Hình 10: Kết quả ghép đôi – Lệnh Telegram và phản hồi của dải LED trên Wokwi]

### 3.3.2. Đánh giá tính ổn định

Hệ thống được đánh giá về tính ổn định qua các tiêu chí sau:

- **Thời gian chạy liên tục:** Hệ thống chạy ổn định hơn 24 giờ trong môi trường mô phỏng và hơn 12 giờ trên phần cứng thực tế mà không bị reset hoặc crash Web Server.
- **Thời gian phản hồi:**
  - Lệnh qua Web (HTTP): độ trễ trung bình dưới 50ms.
  - Lệnh qua giọng nói: thời gian từ lúc nói xong đến khi LED thay đổi màu khoảng 0.8 – 1.5 giây (phụ thuộc vào tốc độ nhận diện của trình duyệt).
  - Lệnh qua Telegram: độ trễ khoảng 1 – 2 giây do thời gian polling của bot.
- **Độ chính xác nhận diện giọng nói:**
  - Trong môi trường yên tĩnh: đạt trên 90% với các lệnh đơn giản.
  - Trong môi trường có tiếng ồn nhẹ: độ chính xác giảm xuống khoảng 75–80%.
- **Khả năng chịu tải:** Hệ thống xử lý tốt nhiều lệnh liên tiếp mà không bị treo hoặc mất kết nối WiFi.
- **Tiêu thụ tài nguyên:**
  - RAM sử dụng khoảng 65% dung lượng khả dụng của ESP32.
  - CPU load trung bình dưới 40% khi không có lệnh.

|                               |         |     |
|-------------------------------|---------|-----|
|                               |         |     |
| <b>Thời gian phản hồi Web</b> | < 50 ms | Tốt |

|                                     |                     |         |
|-------------------------------------|---------------------|---------|
|                                     |                     |         |
| <b>Thời gian phản hồi giọng nói</b> | 0.8 – 1.5s          | Khá     |
| <b>Độ chính xác giọng nói</b>       | 90% (yên tĩnh)      | Tốt     |
| <b>Thời gian chạy liên tục</b>      | > 24 giờ (mô phỏng) | Ổn định |
| <b>Tỷ lệ lỗi kết nối Wifi</b>       | 0% trong 24 giờ     | Rất tốt |

*[Bảng đánh giá hiệu suất hệ thống]*

## PHẦN KẾT LUẬN VÀ KIẾN NGHỊ

### 1. Kết luận

Đề tài “Phát triển hệ thống đèn LED thông minh RGB với ESP32” đã hoàn thành các mục tiêu đề ra. Hệ thống cho phép điều khiển màu sắc và độ sáng của dải LED WS2812 thông qua hai phương thức chính: giao diện Web và nhận diện giọng nói tiếng Việt.

Cụ thể, đề tài đã thực hiện được:

- Thiết kế và triển khai hệ thống điều khiển đèn LED RGB sử dụng vi điều khiển ESP32 và thư viện Adafruit\_NeoPixel.
- Xây dựng Web Server nhúng trên ESP32 với giao diện thân thiện, hỗ trợ chọn màu bằng color picker và điều chỉnh độ sáng.
- Tích hợp thành công tính năng nhận diện giọng nói tiếng Việt sử dụng Web SpeechRecognition API.
- Kết nối và điều khiển hệ thống qua Telegram Bot, cho phép thao tác từ xa.
- Kiểm thử toàn bộ hệ thống trên mô phỏng Wokwi trước khi triển khai thực tế.

Kết quả thực nghiệm cho thấy hệ thống hoạt động ổn định, thời gian phản hồi nhanh và giao diện dễ sử dụng. Việc kết hợp giữa công nghệ IoT, Web và nhận diện giọng nói đã tạo ra một giải pháp chiếu sáng thông minh có tính thực tiễn cao, chi phí thấp và dễ dàng mở rộng.

Đề tài đã góp phần khẳng định khả năng ứng dụng của ESP32 trong việc xây dựng các hệ thống nhà thông minh (Smart Home) đơn giản nhưng hiệu quả.

### 2. Hạn chế

Mặc dù đã đạt được mục tiêu chính, hệ thống vẫn còn một số hạn chế sau:

- Tính năng nhận diện giọng nói chỉ hoạt động khi trình duyệt web đang mở và thiết bị có kết nối mạng.
- Hệ thống hiện chỉ hoạt động tốt trong mạng LAN nội bộ, chưa hỗ trợ điều khiển từ xa qua Internet (Cloud).
- Độ chính xác của nhận diện giọng nói giảm đáng kể trong môi trường có nhiều tiếng ồn.
- Chưa tích hợp các hiệu ứng ánh sáng nâng cao (rainbow, breathing, music sync...).
- Nguồn điện cung cấp cho dải LED WS2812 chưa được tối ưu khi sử dụng số lượng LED lớn.

### 3. Kiến nghị & Hướng phát triển

Để hoàn thiện và nâng cao hệ thống, các hướng phát triển sau được đề xuất:

1. **Tích hợp Cloud và điều khiển từ xa:** Kết nối hệ thống với các nền tảng như Blynk, Firebase hoặc MQTT để có thể điều khiển đèn từ bất kỳ đâu có kết nối Internet.
2. **Cải thiện nhận diện giọng nói:** Sử dụng Google Assistant, Amazon Alexa hoặc tích hợp microphone phần cứng để nâng cao độ chính xác và tính tiện lợi.
3. **Thêm các chế độ hiệu ứng:** Triển khai các hiệu ứng ánh sáng động như cầu vồng (rainbow), fade, chớp nháy theo nhạc.
4. **Tối ưu hóa phần cứng:** Sử dụng nguồn điện riêng ổn định và thêm tụ lọc để hỗ trợ số lượng LED lớn hơn.
5. **Tích hợp cảm biến:** Thêm cảm biến ánh sáng (LDR) hoặc cảm biến chuyển động để hệ thống tự động bật/tắt và điều chỉnh độ sáng theo môi trường.
6. **Giao diện ứng dụng di động:** Phát triển ứng dụng Android/iOS thay vì chỉ dùng Web để tăng tính trải nghiệm người dùng.

Việc tiếp tục nghiên cứu và phát triển đề tài này sẽ góp phần đưa ra một giải pháp chiếu sáng thông minh hoàn chỉnh hơn, có khả năng ứng dụng thực tế trong đời sống.

## TÀI LIỆU THAM KHẢO

- [1] Ashton, K. (2009). That 'Internet of Things' Thing. *RFID Journal*. Truy cập từ: <https://www.itrcorp.com/libraries/RFIDjournal-That%20Internet%20of%20Things%20Thing.pdf> (Bài gốc năm 1999).



- [2] Espressif Systems. (2023). *ESP32 Technical Reference Manual* (Version 5.7). [https://www.espressif.com/sites/default/files/documentation/esp32\\_technical\\_reference\\_manual\\_en.pdf](https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf)
- [3] Espressif Systems. (2025). *ESP32 Arduino Core Documentation*. <https://docs.espressif.com/projects/arduino-esp32/>
- [4] WorldSemi. (2014). *WS2812 Datasheet*. <https://cdn-shop.adafruit.com/datasheets/WS2812.pdf> (Hoặc phiên bản WS2812B: <https://cdn-shop.adafruit.com/datasheets/WS2812B.pdf>)
- [5] Adafruit Industries. (2025). *Adafruit\_NeoPixel Library*. GitHub repository. [https://github.com/adafruit/Adafruit\\_NeoPixel](https://github.com/adafruit/Adafruit_NeoPixel)
- [6] Lough, B. (2024). *UniversalTelegramBot Library*. Arduino Library. <https://github.com/witnessmenow/Universal-Arduino-Telegram-Bot>
- [7] Wokwi. (2025). *Wokwi Documentation - ESP32 Simulation*. <https://docs.wokwi.com/guides/esp32>
- [8] Espressif Systems. (2025). *ESP-IDF Programming Guide*. <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/>
- [9] Adafruit Industries. (2025). *Adafruit\_SSD1306 Library Documentation*. [https://github.com/adafruit/Adafruit\\_SSD1306](https://github.com/adafruit/Adafruit_SSD1306)
- [10] Arduino. (2025). *ArduinoJson Library*. <https://arduinojson.org/>
- [11] FastLED Community. (2025). *FastLED Library* (tùy chọn thay thế cho Adafruit\_NeoPixel). <https://github.com/FastLED/FastLED>
- [12] Mozilla Developer Network. (2025). *Web Speech API - SpeechRecognition*. <https://developer.mozilla.org/en-US/docs/Web/API/SpeechRecognition>
- [13] Telegram. (2025). *Telegram Bot API Documentation*. <https://core.telegram.org/bots/api>
- [14] Nguyễn Văn A. (2025). *Hướng dẫn lập trình ESP32 với Arduino IDE* (Tài liệu nội bộ hoặc sách tham khảo nếu có). Trường Đại học Khoa học, Huế.